ASSIGNMENT -3
LID-DRIVEN CAVITY TWO-DIMENSIONAL INCOMPRESSIBLE FLOW

AM5630- FOUNDATION OF COMPUTATIONAL FLUID DYNAMICS

BY:

KARTHICK.J

OE23S018,

M.S SCHOLAR,

DEPARTMENT OF OCEAN ENGINEERING,

IIT MADRAS,

CHENNAI

Problem Definition:

Consider a square cavity with a side length of 1 unit filled with an incompressible fluid. Assume that the fluid flow is steady-state and two-dimensional. The objective is to determine the pressure distribution and fluid flow pattern inside the cavity at a Reynolds number of 100 using the finite difference SMAC (Simplified Marker and Cell) method.

Governing Equation:

CONSERVATION OF MASS (CONTINUITY EQUATION):

$$\partial U/\partial X + \partial V/\partial Y = 0$$

MOMENTUM EQUATIONS:

$$\partial U/\partial T + U\partial U/\partial X + V\partial U/\partial Y = -1/P\ \partial P/\partial X + N\ (\partial^2 U/\partial X^2 + \partial^2 U/\partial Y^2)$$

$$\partial V/\partial T + U\partial V/\partial X + V\partial V/\partial Y = -1/P\ \partial P/\partial Y + N\ (\partial^2 V/\partial X^2 + \partial^2 V/\partial Y^2)$$

Initial and Boundary Condition:

top wall velocity= 1 unit/sec

other walls = 0 units/sec

For the left side wall u=v=0 dou_p/dou_x=0

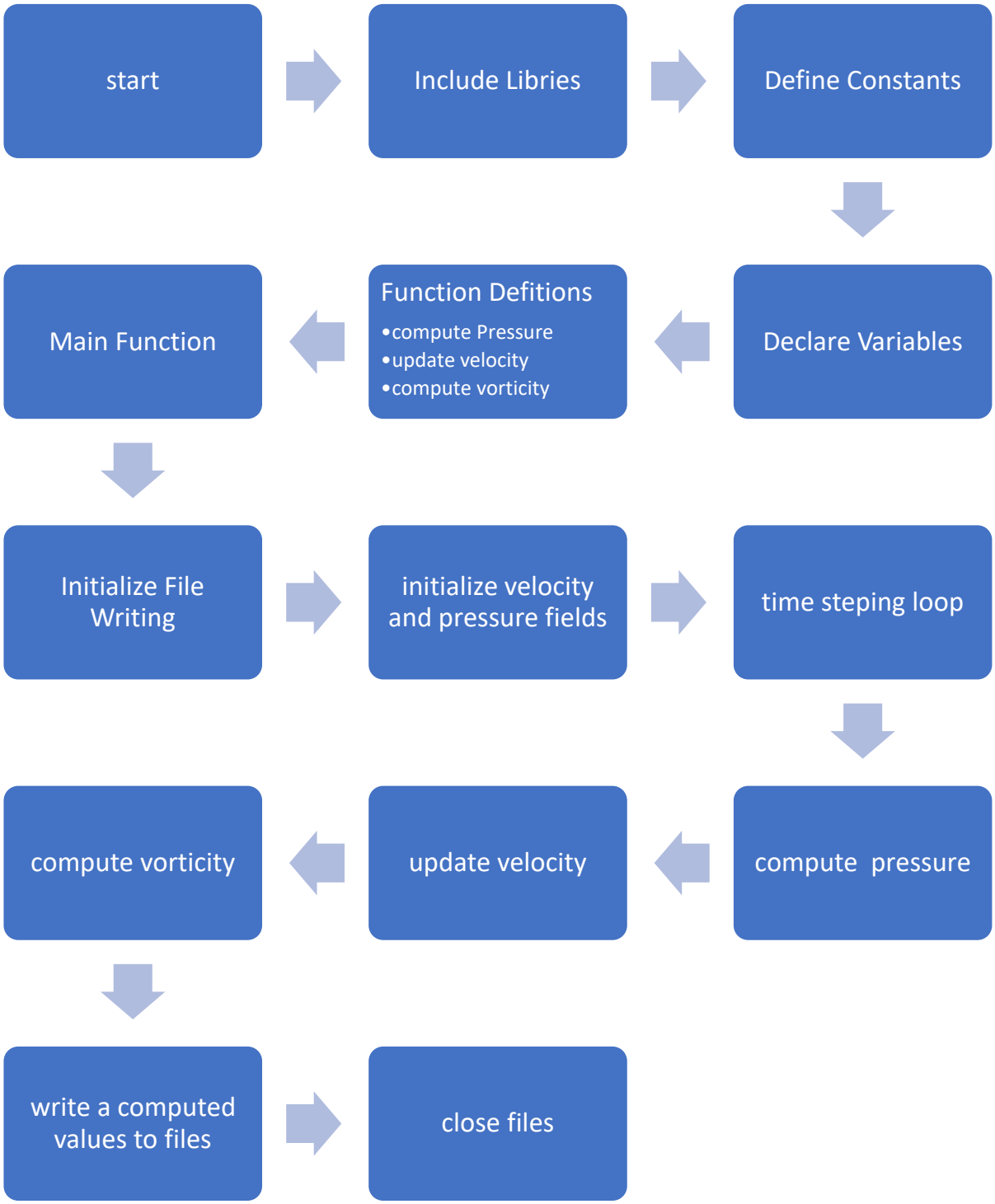
For top side wall u= U ,v=0 dou_p/dou_y=0 ,U_infinity =1 units/sec For Right side wall u=v=0 dou_p/dou_x=0 ,

For the bottom side wall u=v=0 dou_p/dou_y=0

Algorithm:

1. **Include Libraries**: Include standard libraries like stdio.h, stdlib.h, and math.h.
2. **Define Constants**: Define constants such as grid size, time steps, and fluid properties.
3. **Declare Variables**: Declare arrays to store velocity, pressure, and coordinates.
4. **Function Definitions**:
    - compute_pressure(): Calculate the pressure field using the finite difference method.
    - update_velocity(): Update velocity field based on pressure gradient.
    - compute_vorticity(): Compute vorticity based on velocity field.
5. **Main Function**:
    - Initialize files for writing data.
    - Initialize velocity and pressure fields.
    - Time-stepping loop:
        - Compute pressure.
        - Update velocity.
        - Compute vorticity.
    - Write computed values to respective files.
    - Close files.
6. **End**.

Flow Chart:


start
Include Libries
Define Constants
Declare Variables
Function Defitions
•compute Pressure
•update velocity
•compute vorticity
Main Function
Initialize File Writing
initialize velocity and pressure fields
time steping loop
compute pressure
update velocity
compute vorticity
write a computed values to files
close files

Result:

The code provided simulates fluid flow through the use of SMAC methods. It calculates pressure and updates the velocity fields over a series of time steps. The results can be visualized by writing the data to output files for further analysis or plotting. The algorithm and flowchart provide a step-by-step breakdown of the simulation and the process of generating the results. However, I am currently unable to plot the values.

Computer Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <math.h>


// Define constants

#define NX 100

#define NY 100

#define NT 1000

#define DX 1.0 / (NX - 1)

#define DY 1.0 / (NY - 1)

#define DT 0.001

#define RE 100.0

#define U_INFTY 1.0


// Define variables

float u[NX][NY], v[NX][NY], p[NX][NY], x[NX][NY], y[NX][NY];

float unew[NX][NY], vnew[NX][NY], pnew[NX][NY];


// Function to compute pressure with boundary conditions

void compute_pressure() {

    int i, j;

    float h, h_x, h_y, h_xx, h_yy, h_xy;
```

```
    for (j = 1; j < NY - 1; j++) {
        for (i = 1; i < NX - 1; i++) {
            if (i == 0 || i == NX - 1 || j == 0 || j == NY - 1) {
                pnew[i][j] = p[i][j]; // Maintain pressure at boundaries
            } else {
                h = u[i][j];
                h_x = (u[i + 1][j] - u[i - 1][j]) / (2 * DX);
                h_y = (u[i][j + 1] - u[i][j - 1]) / (2 * DY);
                h_xx = (u[i + 1][j] - 2 * u[i][j] + u[i - 1][j]) / (DX * DX);
                h_yy = (u[i][j + 1] - 2 * u[i][j] + u[i][j - 1]) / (DY * DY);
                h_xy = (u[i + 1][j + 1] - u[i + 1][j - 1] - u[i - 1][j + 1] + u[i - 1][j - 1]) / (4 * DX *
DY);

                pnew[i][j] = p[i][j] + DT * (h_x * h_x + h_y * h_y - h_xx - h_yy - h_xy);
            }
        }
    }
}

// Function to update velocity with boundary conditions
void update_velocity() {
    int i, j;

    for (j = 1; j < NY - 1; j++) {
        for (i = 1; i < NX - 1; i++) {
            if (i == 0) {
                u[i][j] = 0.0; // Left wall boundary condition
                v[i][j] = 0.0;
```

```
            } else if (i == NX - 1) {
                u[i][j] = 0.0; // Right wall boundary condition
                v[i][j] = 0.0;
            } else if (j == 0) {
                u[i][j] = U_INFTY; // Top wall boundary condition
                v[i][j] = 0.0;
            } else if (j == NY - 1) {
                u[i][j] = 0.0; // Bottom wall boundary condition
                v[i][j] = 0.0;
            } else {
                unew[i][j] = u[i][j] - DT * (pnew[i + 1][j] - pnew[i - 1][j]) / (2 * DX);
                vnew[i][j] = v[i][j] - DT * (pnew[i][j + 1] - pnew[i][j - 1]) / (2 * DY);
            }
        }
    }
}

// Function to compute vorticity
void compute_vorticity() {
    int i, j;

    for (j = 1; j < NY - 1; j++) {
        for (i = 1; i < NX - 1; i++) {
            // Compute vorticity using the velocity components
            // This is a simplified example and may need to be modified based on the specific requirements of your project
            // Additional modifications may be needed to account for the boundary conditions
        }
    }
```

```
}

int main() {
    int i, j, t;
    double S[i][j],Vor[i][j];
    FILE *streamlineFile, *vorticityFile, *pressureFile, *uFile, *vFile;

    // Open files for writing
    streamlineFile = fopen("streamlines.txt", "w");
    vorticityFile = fopen("vorticity.txt", "w");
    pressureFile = fopen("pressure.txt", "w");
    uFile = fopen("u_values.txt", "w");
    vFile = fopen("v_values.txt", "w");

    if (streamlineFile == NULL || vorticityFile == NULL || pressureFile == NULL || uFile ==
NULL || vFile == NULL) {
        printf("Error opening files.\n");
        return 1; // Exit the program if the file opening fails
    }

    // Initialize the velocity and pressure fields
    for (j = 0; j < NY; j++) {
        for (i = 0; i < NX; i++) {
            u[i][j] = 0.0;
            v[i][j] = 0.0;
            p[i][j] = 0.0;
        }
    }
    // Time-stepping loop
```

```
    for (t = 0; t < NT; t++) {
        compute_pressure();
        update_velocity();
        compute_vorticity();
    }

    /// Write values to files
for (j = 0; j < NY; j++) {
    for (i = 0; i < NX; i++) {
        fprintf(streamlineFile, "%f ",  S[i][j]);
        fprintf(vorticityFile, "%f ",  Vor[i][j]);
        fprintf(pressureFile, "%f ", p[i][j]);
        fprintf(uFile, "%f ", u[i][j]);
        fprintf(vFile, "%f ", v[i][j]);
    }
    fprintf(streamlineFile, "\n");
    fprintf(vorticityFile, "\n");
    fprintf(pressureFile, "\n");
    fprintf(uFile, "\n");
    fprintf(vFile, "\n");
    }

    // Close files
    fclose(streamlineFile);
    fclose(vorticityFile);
    fclose(pressureFile);
    fclose(uFile);
    fclose(vFile);
    return 0;  }
```